

线性回归模型与 Logistic 线性回归模型数据实战

林煜翔¹

目录

1	线性回归模型——某品牌手机价格预测	2
1.1	数据探索与预处理	2
1.2	参数估计与回归关系的显著性检验	6
1.2.1	回归系数的最小二乘估计与回归方程	6
1.2.2	误差方差的最小二乘估计	7
1.2.3	方差分析表	7
1.2.4	回归关系整体显著性检验	8
1.2.5	回归系数显著性与 95% 置信区间	8
1.3	回归诊断	8
1.3.1	残差分析	8
1.3.2	影响性分析	12
1.3.3	多重共线性分析	13
1.4	预测结果	14
1.5	模型改进	15
2	Logistic 线性回归模型——皮马印第安人糖尿病预测	18
2.1	数据探索与预处理	18
2.2	回归系数的估计	20
2.3	回归系数的显著性检验	23
2.3.1	回归系数整体显著性的广义似然比检验	23
2.3.2	单个回归系数显著性的 Wald 检验	24
2.3.3	回归系数的 95% 置信区间	26
2.4	预测结果	27
2.4.1	预测值生成	27
2.4.2	预测性能评估	29
2.5	模型改进	31
3	课后习题 2.2、2.3	35

¹西安交通大学 数学与统计学院 统计 2301 学号 2233514230

1 线性回归模型——某品牌手机价格预测

随着智能手机市场竞争加剧，精准的价格预测对厂商定价策略制定、渠道库存管理及消费者购买决策均具有重要参考价值。手机价格受屏幕尺寸、电池容量、摄像头配置、处理器性能等多维度硬件参数影响，变量间可能存在复杂关联，传统经验定价易出现偏差。

为此，本节以某品牌手机数据集为基础，构建完整的回归分析流程——从数据探索预处理、线性回归建模与诊断，到复共线性问题解决与岭回归优化，旨在通过量化方法识别影响手机价格的关键因素，建立高精度价格预测模型，同时验证线性模型在实际数据中的应用效果与改进路径，为类似电子产品的定价预测提供可复用的分析框架。

数据来源:<https://www.kaggle.com/datasets/mohannapd/mobile-price-prediction>.

1.1 数据探索与预处理

从指定本地路径加载手机数据集 Cellphone.csv，该数据集包含手机价格（响应变量）、产品 ID 以及 12 项硬件配置解释变量；由于产品 ID 仅用于产品标识，故将其移出模型，最终保留 1 个响应变量与 12 个解释变量构成建模数据集。

数据质量检查结果显示，数据集无缺失值，无需额外补值操作；通过摘要统计与范围分析，价格无负数、硬件参数在合理区间内，各变量取值均未发现明显数据异常。

为进一步理解数据特征，本节开展多维度可视化与相关性分析：

> summary(mobile_data)

Product_id		Price		Sale		weight		resolution	
Min.	: 10.0	Min.	: 614	Min.	: 10.0	Min.	: 66.0	Min.	: 1.40
1st Qu.:	237.0	1st Qu.:	1734	1st Qu.:	37.0	1st Qu.:	134.1	1st Qu.:	4.80
Median	: 774.0	Median	: 2258	Median	: 106.0	Median	: 153.0	Median	: 5.15
Mean	: 675.6	Mean	: 2216	Mean	: 621.5	Mean	: 170.4	Mean	: 5.21
3rd Qu.:	1026.0	3rd Qu.:	2744	3rd Qu.:	382.0	3rd Qu.:	170.0	3rd Qu.:	5.50
Max.	: 1339.0	Max.	: 4361	Max.	: 9807.0	Max.	: 753.0	Max.	: 12.20
ppi		cpu.core		cpu.freq		internal.mem		ram	
Min.	: 121.0	Min.	: 0.000	Min.	: 0.000	Min.	: 0.0	Min.	: 0.000
1st Qu.:	233.0	1st Qu.:	4.000	1st Qu.:	1.200	1st Qu.:	8.0	1st Qu.:	1.000
Median	: 294.0	Median	: 4.000	Median	: 1.400	Median	: 16.0	Median	: 2.000
Mean	: 335.1	Mean	: 4.857	Mean	: 1.503	Mean	: 24.5	Mean	: 2.205
3rd Qu.:	428.0	3rd Qu.:	8.000	3rd Qu.:	1.875	3rd Qu.:	32.0	3rd Qu.:	3.000
Max.	: 806.0	Max.	: 8.000	Max.	: 2.700	Max.	: 128.0	Max.	: 6.000
RearCam		Front_Cam		battery		thickness			
Min.	: 0.00	Min.	: 0.000	Min.	: 800	Min.	: 5.100		
1st Qu.:	5.00	1st Qu.:	0.000	1st Qu.:	2040	1st Qu.:	7.600		
Median	: 12.00	Median	: 5.000	Median	: 2800	Median	: 8.400		
Mean	: 10.38	Mean	: 4.503	Mean	: 2842	Mean	: 8.922		
3rd Qu.:	16.00	3rd Qu.:	8.000	3rd Qu.:	3240	3rd Qu.:	9.800		
Max.	: 23.00	Max.	: 20.000	Max.	: 9500	Max.	: 18.500		

图 1: 线性模型各变量数据摘要统计

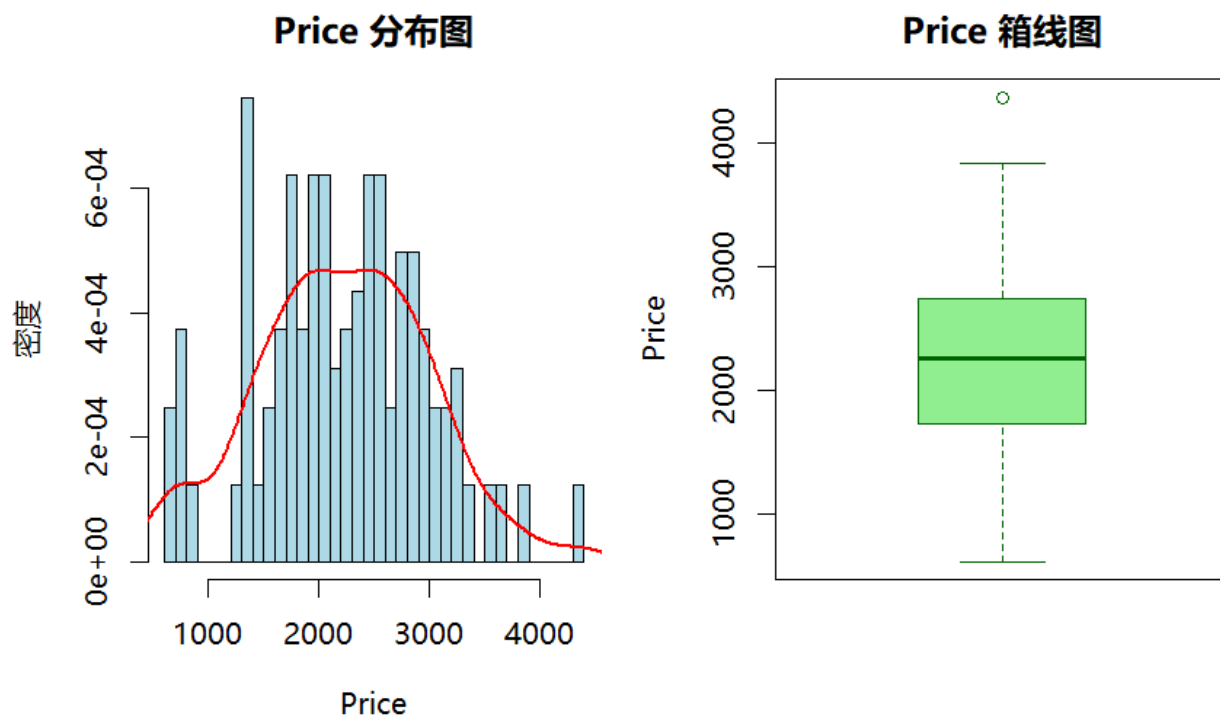


图 2: 线性模型响应变量分布图与箱线图

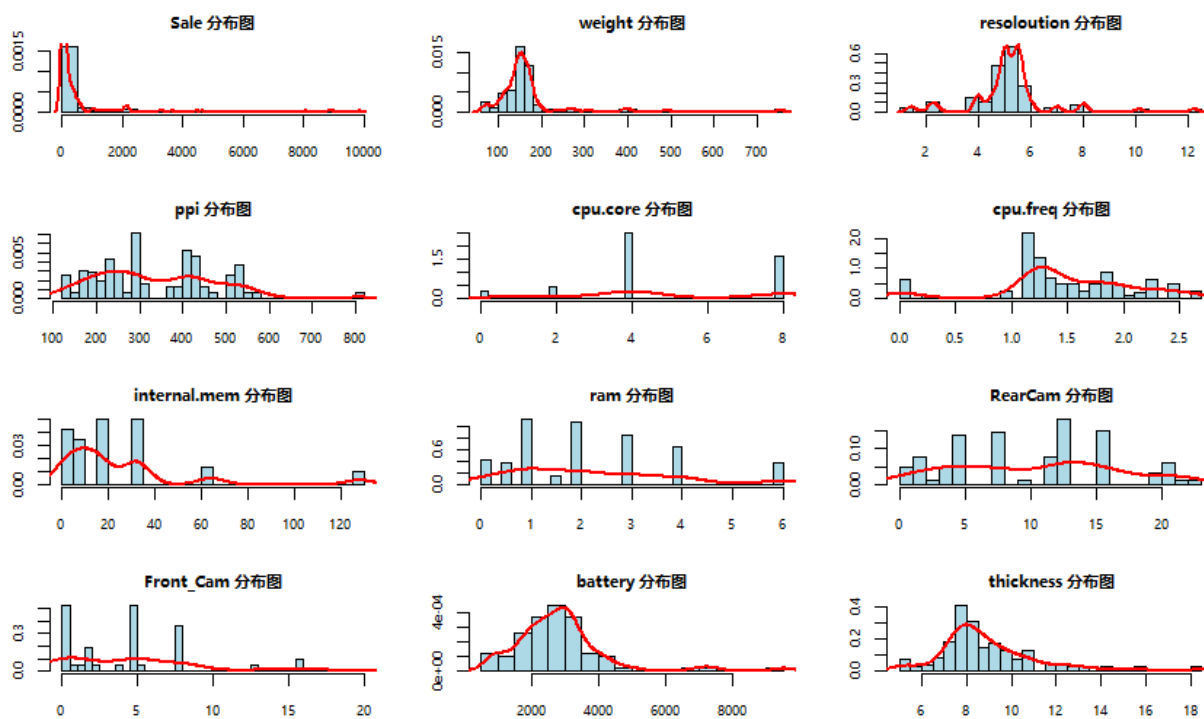


图 3: 线性模型各解释变量分布图

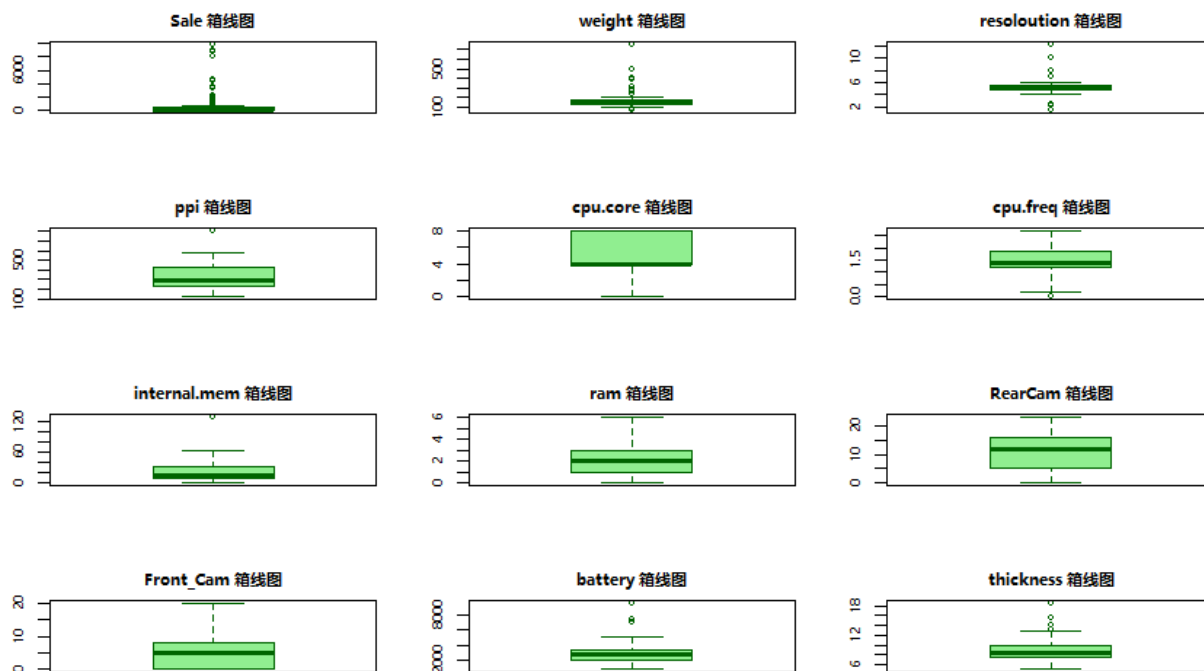


图 4: 线性模型各解释变量箱线图

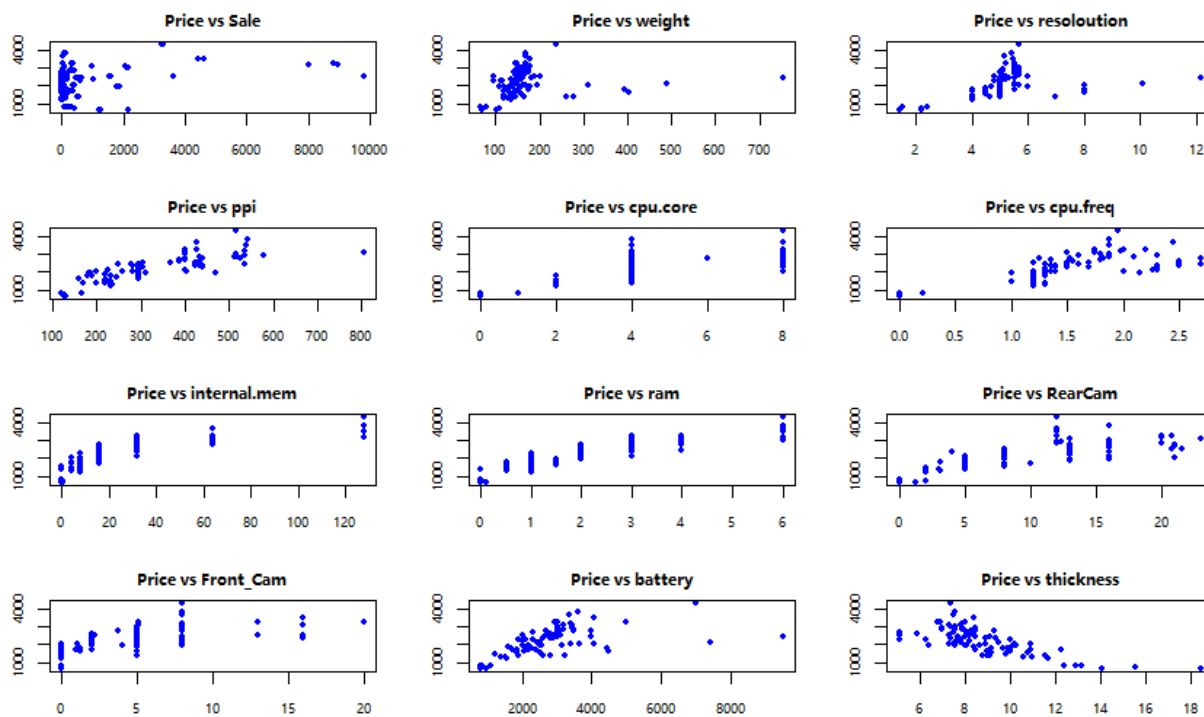


图 5: 线性模型各解释变量与手机价格散点图

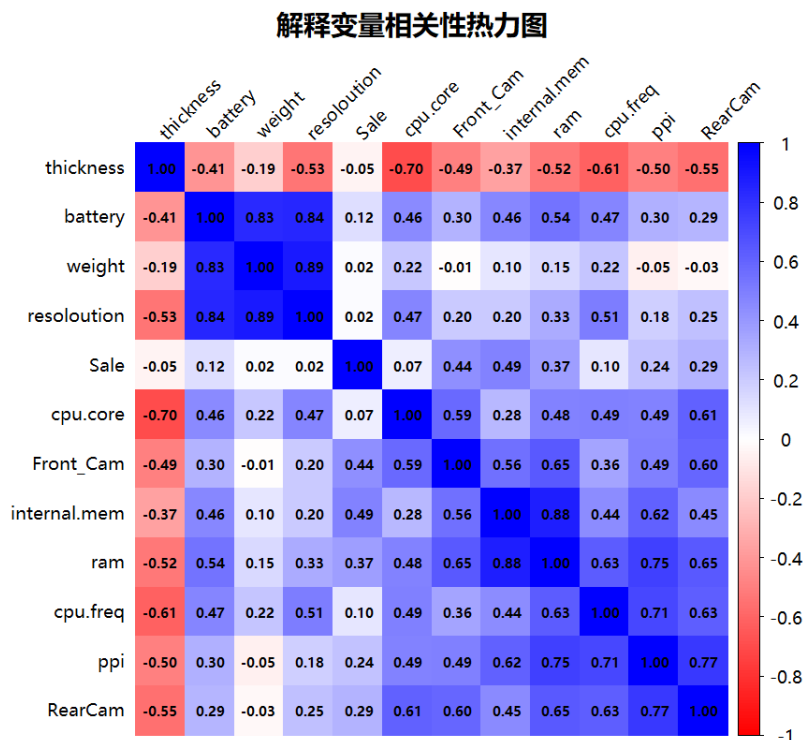


图 6: 线性模型各解释变量相关系数热力图

分析结论如下:

- 通过直方图可见, 各解释变量分布特征各异。“Sale” 存在极大值点, 整体分布严重右偏; “weight”、“resolution”、“battery”、“thickness” 基本呈单峰分布, 也存在一定程度的右偏情况; “cpu.core”、“internal.mem”、“ram”、“RearCam” 和 “Front_Cam” 等分布呈现多峰但峰值集中的特点。响应变量 Price 则呈现近似正态的单峰分布。
- 从箱线图来看, “Sale” 存在明显的异常值, 数据离散程度较高; “weight” 和 “resolution” 的箱体相对较窄, 数据分布较为集中, 且也有较多离群值; “cpu.core” 的箱体上沿达到最大取值, 说明数据在最高段较为集中, “Front_Cam” 的箱体上沿达到最小取值, 说明数据在最低段较为集中; “ppi” “cpu.core” “cpu.freq” “internal.mem” “ram” “RearCam” “battery” 和 “thickness” 的箱体也都有各自的范围。
各解释变量的分布呈现极端值较多的情况, 但这可能是手机销售自身的特点, 如手机品类丰富、单次销量少以及 CPU 核数主流为 8 等, 尚且没有直接的删除或替换方案。
- 从各解释变量与因变量 Price 的散点图可见, 不同变量与 Price 的关联模式各具特点。“Price vs Sale” 的散点较分散, 没有明显规律; “Price vs weight” 在 weight 较低区间数据点更密集, 随 weight 增大 Price 分布逐渐发生变化; “Price vs resolution” 在 resolution 中间取值时, Price 有更集中的较高值分布; “Price vs ppi”、“Price vs cpu.freq”

和“Price vs battery”中可见 Price 与其有较为明显的正相关关系,“Price vs thickness”则显示 Price 与其有较为明显的负相关关系;对于“cpu.core”“internal.mem”“ram”等离散或分档类变量,每个取值下 Price 都有多个数据点分布,有粗糙的正相关关系。

具体而言与 Price 高度相关的变量(按相关性绝对值排序):为 ram: 0.897, ppi: 0.818, internal.mem: 0.777, RearCam: 0.740 以及 cpu.freq: 0.727。

- 从解释变量相关性热力图可知,变量间存在不同程度的相关性:部分变量呈现较强正相关: resolution vs weight: 0.890、battery vs weight: 0.834、weight vs resolution: 0.890、battery vs resolution: 0.843、cpu.freq vs ppi: 0.713;同时, Sale 与多数变量的相关性相对较弱(相关系数绝对值较小)。这些相关性特征为后续分析多重共线性、构建模型等提供了重要参考。

1.2 参数估计与回归关系的显著性检验

1.2.1 回归系数的最小二乘估计与回归方程

```
> n <- nrow(data_for_regression)
> p <- length(coef(lm_model))
> predictor_names <- names(data_for_regression)[names(data_for_regression) != "Price"]
>
> cat("\n1. 回归系数的最小二乘估计与回归方程: \n")

1. 回归系数的最小二乘估计与回归方程:
> coef_vector <- coef(lm_model)
> coef_df <- data.frame(变量 = names(coef_vector), 系数估计 = round(coef_vector, 6))
> print(coef_df, row.names = FALSE)
      变量      系数估计
(Intercept) 1749.550929
      Sale    -0.022747
      weight  -0.379634
resolution  -77.592031
      ppi      1.005718
      cpu.core 53.980891
      cpu.freq 131.891648
internal.mem   6.249926
      ram     93.203185
      RearCam  5.045287
      Front_Cam 8.968397
      battery  0.125890
      thickness -73.688822
>
> eq_terms <- paste0(round(coef_vector[-1], 4), " * ", predictor_names)
> reg_eq <- paste("Price_hat = ", round(coef_vector[1], 4), "+", paste(eq_terms, collapse = " + "))
> cat("回归方程: \n")
      回归方程:
> cat("", reg_eq, "\n")
```

```
Price_hat = 1749.5509 + -0.0227 * Sale + -0.3796 * weight + -77.592 * resolution + 1.0057 *
  ppi + 53.9809 * cpu.core + 131.8916 * cpu.freq + 6.2499 * internal.mem + 93.2032 * ram +
  5.0453 * RearCam + 8.9684 * Front_Cam + 0.1259 * battery + -73.6888 * thickness
```

1.2.2 误差方差的最小二乘估计

```
> residuals_lm <- resid(lm_model)
> rss <- sum(residuals_lm^2)
> sigma2_hat <- rss / lm_model$df.residual
> cat(sprintf("\n2. 误差方差估计: \sigma^2=%.4f (RSS=%.4f, 自由度=%d)\n",
+           sigma2_hat, rss, lm_model$df.residual))
```

2. 误差方差估计: $\sigma^2 = 29295.1994$ (RSS = 4335689.5043, 自由度 = 148)

1.2.3 方差分析表

```
> cat("\n3. 方差分析表: \n")
3. 方差分析表:
> # 提取方差分析数据
> anova_data <- anova(lm_model)
>
> # 计算回归平方和、自由度和均方
> ssr <- sum(anova_data$'Sum Sq'[-nrow(anova_data)])
> dfr <- sum(anova_data$Df[-nrow(anova_data)])
> msr <- ssr / dfr
>
> # 计算误差平方和、自由度和均方
> sse <- anova_data$'Sum Sq'[nrow(anova_data)]
> dfe <- anova_data$Df[nrow(anova_data)]
> mse <- sse / dfe
>
> # 计算总平方和和总自由度
> ss_total <- ssr + sse
> df_total <- dfr + dfe
>
> # 计算F统计量和p值
> f_stat <- msr / mse
> p_value <- pf(f_stat, dfr, dfe, lower.tail = FALSE)
>
> # 创建符合用户要求的方差分析表
> variance_table <- data.frame(
+   "方差来源" = c("回归(R)", "误差(E)", "总和(T)"),
+   "自由度" = c(dfr, dfe, df_total),
+   "平方和(SS)" = c(ssr, sse, ss_total),
+   "均方(MS)" = c(msr, mse, NA),
+   "F值" = c(f_stat, NA, NA),
+   "p值" = c(p_value, NA, NA)
+ )
>
```

```
> # 格式化输出表格
> print("表2.1方差分析表")
[1] "表2.1方差分析表"
> print(format(variance_table, digits = 6, nsmall = 4, na.encode = FALSE), row.names = FALSE)
```

方差来源	自由度	平方和.SS.	均方.MS.	F值	p值
回归(R)	12	90082155.2534	7506846.2711	256.2483	1.73991e-92
误差(E)	148	4335689.5043	29295.1994	NA	NA
总和(T)	160	94417844.7578	NA	NA	NA

1.2.4 回归关系整体显著性检验

```
> summary_lm <- summary(lm_model)
> f_stat <- summary_lm$fstatistic
> f_p_value <- pf(f_stat[1], f_stat[2], f_stat[3], lower.tail = FALSE)
> cat(sprintf("\n4. 回归关系整体显著性检验: F(%d,%d)=%.4f, p-value=%.6g\n",
+           f_stat[2], f_stat[3], f_stat[1], f_p_value))
```

4. 回归关系整体显著性检验: F(12, 148) = 256.2483, p-value = 1.73991e-92

1.2.5 回归系数显著性与 95% 置信区间

```
> cat("\n5. 回归系数显著性与95%置信区间: \n")

5. 回归系数显著性与95%置信区间:
> coef_table <- summary_lm$coefficients
> ci_table <- confint(lm_model, level = 0.95)
> coef_summary <- cbind(round(coef_table, 6), round(ci_table, 6))
> colnames(coef_summary)[5:6] <- c("CI_2.5%", "CI_97.5%")
> print(coef_summary)
```

	Estimate	Std. Error	t value	Pr(> t)	CI_2.5%	CI_97.5%
(Intercept)	1749.550929	215.926030	8.102548	0.000000	1322.854644	2176.247213
Sale	-0.022747	0.011610	-1.959205	0.051968	-0.045690	0.000196
weight	-0.379634	0.694179	-0.546882	0.585283	-1.751416	0.992148
resolution	-77.592031	42.253453	-1.836348	0.068313	-161.090030	5.905969
ppi	1.005718	0.217058	4.633413	0.000008	0.576785	1.434651
cpu.core	53.980891	9.757502	5.532245	0.000000	34.698872	73.262909
cpu.freq	131.891648	44.527837	2.962004	0.003562	43.899189	219.884108
internal.mem	6.249926	1.225476	5.100000	0.000001	3.828236	8.671616
ram	93.203185	25.618878	3.638067	0.000379	42.577144	143.829225
RearCam	5.045287	4.331663	1.164746	0.245996	-3.514610	13.605183
Front_Cam	8.968397	5.194611	1.726481	0.086348	-1.296790	19.233584
battery	0.125890	0.030408	4.140021	0.000058	0.065800	0.185980
thickness	-73.688822	12.475288	-5.906783	0.000000	-98.341519	-49.036124

1.3 回归诊断

1.3.1 残差分析

```

> cat("\n1. 残差分析: \n")

1. 残差分析:
> studentized_residuals <- rstandard(lm_model)
> old_par <- par(no.readonly = TRUE)
> windowsFonts(微软雅黑 = windowsFont("微软雅黑"))
> par(family = "微软雅黑")
>
> # 创建QQ图并计算相关系数
> qq_obj <- qqnorm(studentized_residuals,
+   main = "学生化残差的正态QQ图",
+   xlab = "理论分位数",
+   ylab = "样本分位数",
+   pch = 19,
+   col = "blue")
> qqline(studentized_residuals, col = "red", lwd = 2)
>
> # 提取QQ图中的理论分位数和样本分位数
> theoretical_quantiles <- qq_obj$x
> sample_quantiles <- qq_obj$y
>
> # 计算理论分位数和样本分位数之间的相关系数
> qq_correlation <- cor(theoretical_quantiles, sample_quantiles)
> cat(sprintf("QQ图相关性系数: r = %.4f\n", qq_correlation))
  QQ图相关性系数: r = 0.9935
>
> # Shapiro-Wilk正态性检验
> shapiro_sample <- studentized_residuals
> if (length(shapiro_sample) > 5000) {
+   set.seed(2025)
+   shapiro_sample <- sample(shapiro_sample, 5000)
+   cat("样本量>5000, 已抽样5000个残差用于Shapiro-Wilk检验\n")
+ }
> shapiro_result <- shapiro.test(shapiro_sample)
> cat(sprintf("Shapiro-Wilk正态性检验: W = %.4f, p-value = %.6g\n",
+   shapiro_result$statistic, shapiro_result$p.value))
  Shapiro-Wilk正态性检验: W = 0.9863, p-value = 0.114983
>
> # 综合判断残差是否服从正态分布
> cat("正态分布结论: ")
  正态分布结论: > if (shapiro_result$p.value > 0.05 && qq_correlation > 0.99) {
+   cat("残差高度服从正态分布 (p > 0.05, r > 0.99)\n")
+ } else if (shapiro_result$p.value > 0.05 && qq_correlation > 0.95) {
+   cat("残差基本服从正态分布 (p > 0.05, r > 0.95)\n")
+ } else if (shapiro_result$p.value <= 0.05 && qq_correlation > 0.98) {
+   cat("残差近似服从正态分布 (p <= 0.05, r > 0.98), 轻微偏离\n")
+ } else {
+   cat("残差不服从正态分布 (p <= 0.05, r <= 0.98), 存在明显偏离\n")
+ }
  残差高度服从正态分布 (p > 0.05, r > 0.99)
>
> hist(studentized_residuals,

```

```

+     breaks = "FD",
+     main = "学生化残差直方图",
+     xlab = "学生化残差",
+     ylab = "频数",
+     col = "lightblue",
+     border = "black")
> lines(density(studentized_residuals, na.rm = TRUE), col = "red", lwd = 2)
>
> cat("\n2. 残差图分析 (线性性与方差齐性) : \n")

2. 残差图分析 (线性性与方差齐性) :
> predicted_values <- predict(lm_model)
> plot(predicted_values, studentized_residuals,
+     main = "学生化残差 vs 拟合值",
+     xlab = "拟合值",
+     ylab = "学生化残差",
+     pch = 19,
+     col = "purple")
> abline(h = 0, col = "red", lwd = 2)
> abline(h = c(-2, 2), col = "orange", lty = 2)
> sig_vars <- rownames(coef_table)[which(coef_table[, 4] < 0.05)]
> sig_vars <- setdiff(sig_vars, "(Intercept)")
> if (length(sig_vars) > 0) {
+   n_cols <- 3
+   n_rows <- ceiling(length(sig_vars) / n_cols)
+   par(mfrow = c(n_rows, n_cols))
+   par(mar = c(3.5, 3.5, 2, 1))
+   for (var in sig_vars) {
+     plot(data_for_regression[[var]], studentized_residuals,
+         main = paste("残差 vs", var),
+         xlab = var,
+         ylab = "学生化残差",
+         pch = 19,
+         col = "darkred")
+     abline(h = 0, col = "gray40", lwd = 1)
+   }
+   par(old_par)
+ }

```

- 残差正态性检验通过 QQ 图与直方图实现：学生化残差的 QQ 图近似贴合对角线，残差与理论正态分位数的相关系数 $\hat{\rho} = 0.993535 > 0.99$ ，表明残差符合正态分布；

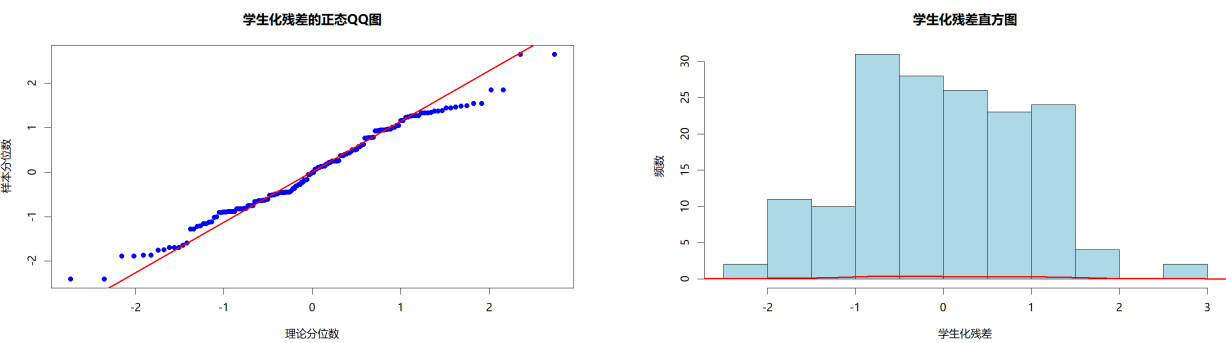


图 7: 学生化残差的正态性检验

- 残差与响应变量的关系图显示，散点大致在一个以横轴为中心的带状区域内，且不呈现任何的趋势，从而佐证了残差的正态性。

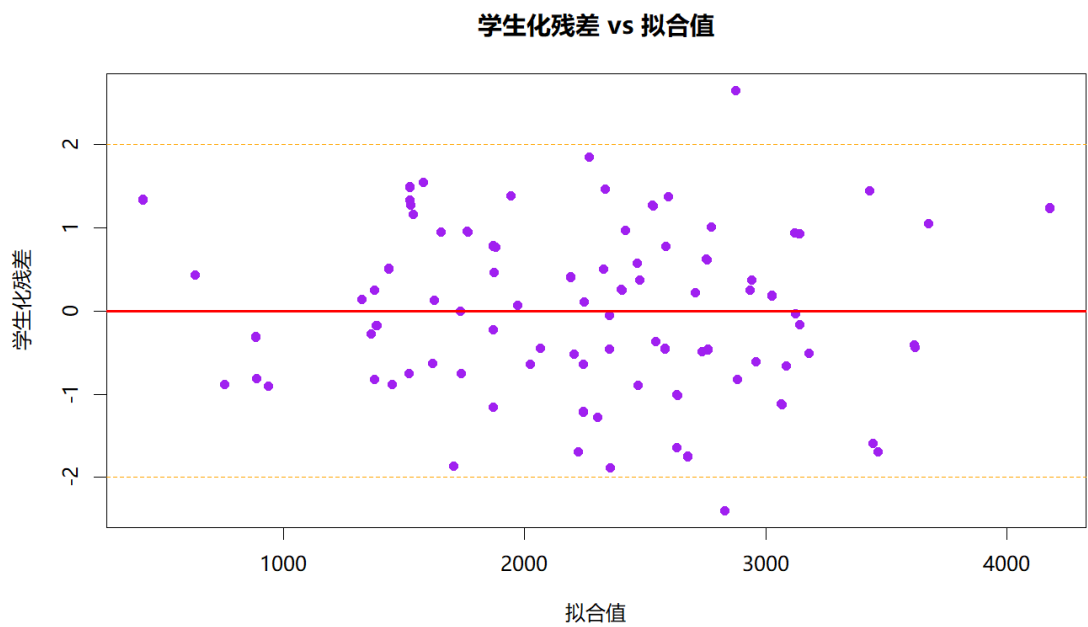


图 8: 线性模型残差-响应变量散点图

- 残差与解释变量的散点图显示，对大部分解释变量而言散点大致在一个以横轴为中心的带状区域内，且不呈现任何的趋势，从而也佐证了残差的正态性。

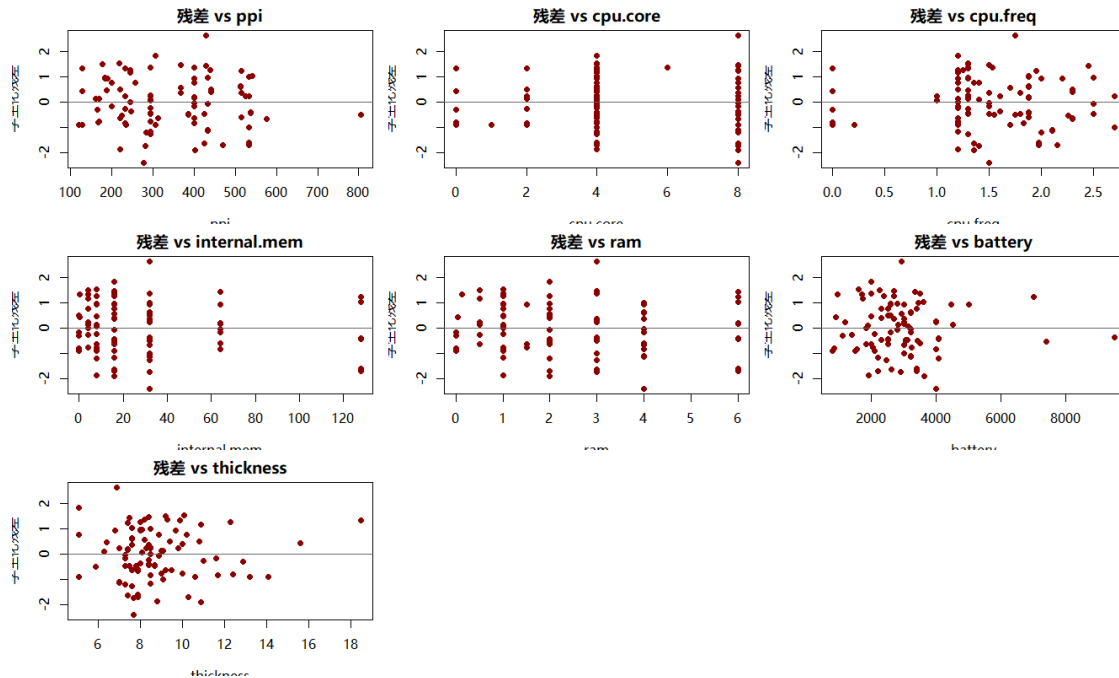


图 9: 线性模型残差-解释变量散点图

1.3.2 影响性分析

```
> cat("\n3. 影响性分析: \n")

3. 影响性分析:
> leverage_vals <- hatvalues(lm_model)
> leverage_threshold <- 2 * p / n
> high_leverage_idx <- which(leverage_vals > leverage_threshold)
> cat(sprintf("杠杆统计量阈值: %.4f, 高杠杆点数量: %d\n",
+             leverage_threshold, length(high_leverage_idx)))
杠杆统计量阈值: 0.1615, 高杠杆点数量: 12
>
> covratio_vals <- covratio(lm_model)
> covratio_lower <- 1 - 3 * p / n
> covratio_upper <- 1 + 3 * p / n
> covratio_flags <- which(covratio_vals < covratio_lower | covratio_vals > covratio_upper)
> cat(sprintf("Covratio 判断区间: [%.4f, %.4f], 异常点数量: %d\n",
+             covratio_lower, covratio_upper, length(covratio_flags)))
Covratio 判断区间: [0.7578, 1.2422], 异常点数量: 18
>
> influential_points <- sort(unique(c(high_leverage_idx, covratio_flags)))
> if (length(influential_points) > 0) {
+   cat("需要关注的影响性观测编号:\n")
+   print(head(influential_points, 20))
+ } else {
+   cat("未检测到显著影响点\n")
+ }

需要关注的影响性观测编号:
```

```
[1] 11 21 33 38 41 48 84 88 128 129 147 148 149 152 153 154 156 157 158 159
>
> plot(leverage_vals,
+      main = "杠杆值分布",
+      ylab = "Leverage",
+      xlab = "观测编号",
+      pch = 19,
+      col = "steelblue")
> abline(h = leverage_threshold, col = "red", lty = 2)
>
> plot(covratio_vals,
+      main = "Covratio分布",
+      ylab = "Covratio",
+      xlab = "观测编号",
+      pch = 19,
+      col = "darkgreen")
> abline(h = c(covratio_lower, covratio_upper), col = "orange", lty = 2)
```

从下图可以看出，样本中存在少量强影响点（基准线之上），可考虑在后续回归中剔除。

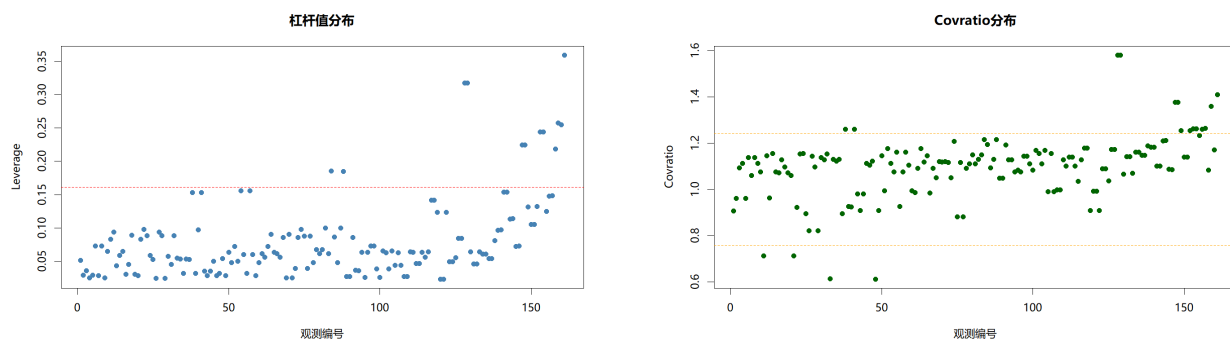


图 10: 基于杠杆统计量与 Covratio 统计量样本的影响性分析

1.3.3 多重共线性分析

```
> cat("\n4. 多重共线性分析（条件数方法）：\n")

4. 多重共线性分析（条件数方法）：
> x_matrix_scaled <- scale(data_for_regression[, predictor_names])
> condition_number <- kappa(x_matrix_scaled, exact = TRUE)
> cat(sprintf("条件数 \kappa = %.4f\n", condition_number))
条件数 \kappa = 16.2417
> if (condition_number > 1000) {
+   cat("\kappa > 100, 表明存在严重多重共线性\n")
+ } else if (condition_number > 100) {
+   cat("100 < \kappa <= 1000, 表明存在明显多重共线性\n")
+ } else {
+   cat("\kappa <= 100, 多重共线性问题可接受\n")
+ }
\kappa <= 100, 多重共线性问题可接受
```

1.4 预测结果

```

> set.seed(456)
> train_indices <- sample(seq_len(nrow(data_for_regression)), size = ceiling(0.7 * nrow(data_for_
  regression)))
> train_data <- data_for_regression[train_indices, ]
> test_data <- data_for_regression[-train_indices, ]
>
> lm_train <- lm(Price ~ ., data = train_data)
> test_pred <- predict(lm_train, newdata = test_data)
> test_conf_int <- predict(lm_train, newdata = test_data, interval = "confidence", level = 0.95)
> test_pred_int <- predict(lm_train, newdata = test_data, interval = "prediction", level = 0.95)
> test_actual <- test_data$Price
>
> cat("\n测试集预测值点估计与区间估计 (前10条) : \n")

  测试集预测值点估计与区间估计 (前10条) :
> prediction_summary <- data.frame(
+   实际值 = round(test_actual[seq_len(min(10, length(test_actual)))], 2),
+   点预测 = round(test_pred[seq_len(min(10, length(test_pred)))], 2),
+   置信下限 = round(test_conf_int[seq_len(min(10, nrow(test_conf_int)))], "lwr", 2),
+   置信上限 = round(test_conf_int[seq_len(min(10, nrow(test_conf_int)))], "upr", 2),
+   预测下限 = round(test_pred_int[seq_len(min(10, nrow(test_pred_int)))], "lwr", 2),
+   预测上限 = round(test_pred_int[seq_len(min(10, nrow(test_pred_int)))], "upr", 2)
+ )
> print(prediction_summary, row.names = FALSE)
  实际值 点预测 置信下限 置信上限 预测下限 预测上限
1749 1558.65 1491.32 1625.99 1222.18 1895.12
1916 2048.97 1971.83 2126.11 1710.40 2387.53
2438 2955.51 2832.45 3078.57 2603.63 3307.39
2938 2775.21 2660.33 2890.10 2426.11 3124.32
1612 1738.52 1660.49 1816.55 1399.75 2077.29
2438 2955.37 2832.32 3078.43 2603.49 3307.26
1390 1721.92 1659.04 1784.79 1386.31 2057.52
2977 3146.84 3028.72 3264.97 2796.66 3497.03
1777 1539.81 1464.69 1614.94 1201.70 1877.93
2087 2371.57 2297.34 2445.79 2033.65 2709.48
>
> calc_metrics <- function(actual, pred) {
+   mse_val <- mean((pred - actual)^2)
+   rmse_val <- sqrt(mse_val)
+   mae_val <- mean(abs(pred - actual))
+   r2_val <- 1 - sum((pred - actual)^2) / sum((actual - mean(actual))^2)
+   data.frame(复决定系数 = round(r2_val, 4),
+             MSE = round(mse_val, 4),
+             RMSE = round(rmse_val, 4),
+             MAE = round(mae_val, 4))
+ }
>
> train_metrics <- calc_metrics(train_data$Price, predict(lm_train, newdata = train_data))
> test_metrics <- calc_metrics(test_actual, test_pred)
>

```

```

> cat("训练集性能指标: \n")
训练集性能指标:
> print(train_metrics, row.names = FALSE)
复决定系数      MSE      RMSE      MAE
0.9573 24433.8 156.3131 127.545
> cat("\n测试集性能指标: \n")

测试集性能指标:
> print(test_metrics, row.names = FALSE)
复决定系数      MSE      RMSE      MAE
0.9274 41677.53 204.1508 171.1262
>
> overall_r2 <- summary_lm$r.squared
> overall_mae <- mean(abs(predicted_values - data_for_regression$Price))
> overall_mse <- mean((predicted_values - data_for_regression$Price)^2)
> cat(sprintf("\n全量样本: R²=□%.4f, □MSE=□%.4f, □MAE=□%.4f\n",
+             overall_r2, overall_mse, overall_mae))

全量样本: R² = 0.9541, MSE = 26929.7485, MAE = 136.2621

```

1.5 模型改进

```

> needs_boxcox <- !is.na(shapiro_result$p.value) && shapiro_result$p.value < 0.05
> needs_influence_removal <- length(influential_points) > 0
> needs_ridge <- condition_number > 30
>
> improved_df <- data_for_regression
> improved_df$Response_for_model <- improved_df$Price
> response_label <- "Price"
> lambda_opt <- NA
>
> if (needs_boxcox) {
+   cat("\n1. □残差正态性未通过, 执行Box-Cox变换: \n")
+   lambda_seq <- seq(-2, 2, by = 0.1)
+   boxcox_result <- boxcox(lm_model, lambda = lambda_seq, plotit = TRUE,
+                           main = "Box-Cox □ 选择图")
+   lambda_opt <- boxcox_result$x[which.max(boxcox_result$y)]
+   cat(sprintf("Box-Cox最优 □=□%.3f\n", lambda_opt))
+   if (abs(lambda_opt) < 1e-6) {
+     improved_df$Response_for_model <- log(improved_df$Price)
+     response_label <- "log(Price)"
+     cat("采用对数变换 ( \approx 0) \n")
+   } else {
+     improved_df$Response_for_model <- (improved_df$Price^lambda_opt - 1) / lambda_opt
+     response_label <- sprintf("BoxCox(Price, □ =%.2f)", lambda_opt)
+   }
+ } else {
+   cat("\n1. 差正态性可接受, 暂不进行Box-Cox变换\n")
+ }

```

1. 残差正态性可接受, 暂不进行Box-Cox变换

```
>
> if (needs_influence_removal) {
+   cat("\n2.检测到影响性观测, 执行剔除: \n")
+   improved_df <- improved_df[-influential_points, ]
+   cat(sprintf("已剔除%d个观测, 剩余样本量:%d\n",
+               length(influential_points), nrow(improved_df)))
+ } else {
+   cat("\n2.未检测到显著影响点, 无需剔除\n")
+ }
```

2. 检测到影响性观测, 执行剔除:

已剔除 22 个观测, 剩余样本量: 139

```
>
> model_data_final <- improved_df[, c("Response_for_model", predictor_names)]
> colnames(model_data_final)[1] <- "Y_improved"
>
> improved_lm_model <- lm(Y_improved ~ ., data = model_data_final)
> cat("\n3.改进后普通回归模型结果: \n")
```

3. 改进后普通回归模型结果:

```
> print(summary(improved_lm_model)$coefficients)
              Estimate   Std. Error    t value    Pr(>|t|)
(Intercept)  1.639611e+03  213.05510451   7.695713534 3.547838e-12
Sale         1.338361e-04   0.03038776   0.004404276 9.964929e-01
weight       -8.409467e-01   0.82548111  -1.018735262 3.102808e-01
resolution  -4.617005e+01  47.77047085  -0.966497695 3.356466e-01
ppi          1.010056e+00   0.27667996   3.650628539 3.819283e-04
cpu.core     4.863837e+01   9.90983278   4.908091664 2.787489e-06
cpu.freq     1.196320e+02  45.78512023   2.612901369 1.006987e-02
internal.mem 3.751349e+00   2.04134107   1.837688615 6.846407e-02
ram          1.298485e+02  32.11343760   4.043431695 9.113447e-05
RearCam      6.330991e-01   4.44659333   0.142378468 8.870085e-01
Front_Cam    8.895825e+00   5.69438676   1.562209497 1.207468e-01
battery      1.434490e-01   0.03817117   3.758045638 2.607134e-04
thickness    -7.015312e+01  12.27408545  -5.715547701 7.489419e-08
>
> if (!is.na(lambda_opt)) {
+   cat(sprintf("该模型的因变量为%s\n", response_label))
+ }
>
> if (needs_ridge) {
+   cat("\n4.条件数提示复共线性, 需要实施岭回归: \n")
+   x_improved <- as.matrix(model_data_final[, -1])
+   y_improved <- model_data_final$Y_improved
+   set.seed(123)
+   cv_ridge <- cv.glmnet(x_improved, y_improved, alpha = 0, nfolds = 10)
+   lambda_min <- cv_ridge$lambda.min
+   lambda_1se <- cv_ridge$lambda.1se
+   cat(sprintf(" .min=%%.6f, .1se=%%.6f\n", lambda_min, lambda_1se))
+   ridge_model_min <- glmnet(x_improved, y_improved, alpha = 0, lambda = lambda_min)
+   ridge_coef <- as.matrix(coef(ridge_model_min))
```

```
+   cat("岭回归系数 ( .min) : \n")
+   print(round(ridge_coef, 5))
+   ridge_pred <- predict(ridge_model_min, newx = x_improved)
+   ridge_mse <- mean((ridge_pred - y_improved)^2)
+   ridge_mae <- mean(abs(ridge_pred - y_improved))
+   ridge_r2 <- 1 - sum((ridge_pred - y_improved)^2) / sum((y_improved - mean(y_improved))^2)
+   cat(sprintf("改进后岭回归性能: R²=□%.4f, □MSE=□%.4f, □MAE=□%.4f\n",
+               ridge_r2, ridge_mse, ridge_mae))
+ } else {
+   cat("\n4.条件数无明显异常, 无需岭回归\n")
+ }
```

4. 条件数无明显异常, 无需岭回归

2 Logistic 线性回归模型——皮马印第安人糖尿病预测

皮马印第安人是美洲原住民的一个分支，是全球糖尿病高发人群之一，其遗传背景与生活习惯导致该群体糖尿病患病率显著高于普通人群，早期风险筛查对疾病防控具有重要意义。糖尿病的诊断依赖血糖、血压、BMI 等医学指标，这些指标与患病状态存在复杂的非线性关联，传统经验判断易出现偏差。广义线性模型中的 Logistic 线性回归即二项分布响应变量 +logit 典则连接函数是处理二分类预测任务的经典工具，既能输出患病概率，又能量化各指标对患病风险的影响程度，具备良好的可解释性。

为此，本研究以皮马印第安人糖尿病数据集为基础，构建完整的 Logistic 线性回归建模流程——从医学数据预处理、模型拟合与多维度诊断，到逐步回归特征选择优化，旨在识别影响糖尿病发病的关键医学指标，建立高精度的患病预测模型，为该群体的糖尿病早期筛查提供量化工具，同时验证广义线性模型在医学二分类任务中的实用性与优化路径。

数据来源：<https://www.kaggle.com/datasets/kandij/diabetes-dataset>。该数据由美国国家糖尿病、消化与肾脏疾病研究所 (*National Institute of Diabetes and Digestive and Kidney Diseases*) 收集并公开，是皮马印第安人糖尿病数据集 (*Pima Indians Diabetes Database*) 的一部分，其中所有受试者均属于皮马印第安人，且为 21 岁及以上女性。

2.1 数据探索与预处理

本节首先完成数据加载与医学数据预处理。加载皮马印第安人糖尿病数据集 `diabetes2.csv`，该数据集包含血糖 Glucose、血压 BloodPressure、BMI、皮肤厚度 SkinThickness、胰岛素 Insulin、年龄 Age、糖尿病家族史 Pregnancies、血糖浓度 DiabetesPedigreeFunction 等 8 个解释变量与 1 个二分类因变量 Outcome (1 患糖尿病，0 未患)，原始样本量为 768 例。

医学数据的核心预处理在于“零值处理”——部分变量的零值并非真实测量值，而是缺失数据的替代：其中血糖、血压、BMI 在医学上不可能为 0，此类零值直接导致样本无效，故删除含这些零值的行，样本量降至约 600 例；皮肤厚度的零值占比适中，若删除会丢失过多信息，故用非零值的均值替换零值，既保留样本完整性，又减少缺失数据的干扰。

数据探索通过多维度可视化与统计分析展开：

- 各解释变量的分布图显示，怀孕次数 Pregnancies 是离散值，在低值区间 0 - 5 次左右分布更为密集，整体呈右偏形态；葡萄糖 Glucose 的分布接近单峰，在 100 左右形成集中区域，近似正态分布；血压 BloodPressure 主要集中在 60 - 80 的区间内，分布接近正态；皮肤厚度 SkinThickness 在 20 - 40 附近有显著峰值；胰岛素 Insulin 在低值区域占比较高，呈现明显右偏；身体质量指数 BMI 以 30 左右为中心形成单峰，分布略右偏；糖尿病遗传函数 DiabetesPedigreeFunction 在 0 - 0.5 区间分布更多，呈右偏分布；年龄 Age 在 20 - 30 岁左右分布较为集中，整体呈右偏形态。

- 从箱线图来看, Pregnancies 的箱体集中于低数值区间, 存在少量高值异常点, 反映多数样本怀孕次数较少但部分样本偏高; Glucose 的箱体较为集中且异常值少, 分布相对稳定; BloodPressure 的箱体中间集中、波动范围不大, 但异常值较多; SkinThickness 的箱体有一定集中性, 同时存在高值异常点; Insulin 的箱体以低数值为主, 却有明显的高值异常点; BMI 的箱体分布集中、数值范围稳定, 异常值多存在上方区域; DiabetesPedigreeFunction 的箱体集中在低区间, 存在少量高值异常点; Age 的箱体显示多数样本年龄集中在一定范围, 同时存在少量高龄的异常点。

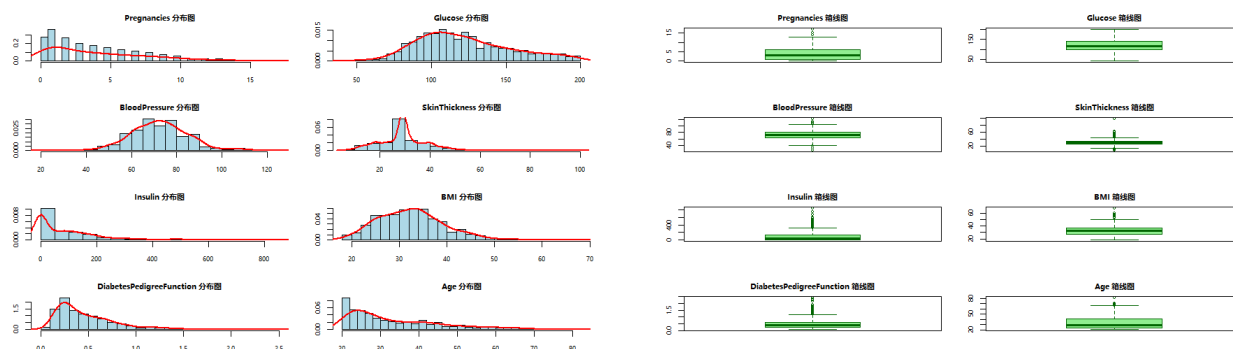


图 11: Logistic 线性回归各解释变量分布与箱线图

- 从解释变量相关性热力图可知, 部分变量间存在少量的线性关联: Pregnancies 与 Age 的相关系数为 0.56, SkinThickness 与 BMI 的相关系数为 0.55, 均呈现中等的正相关。

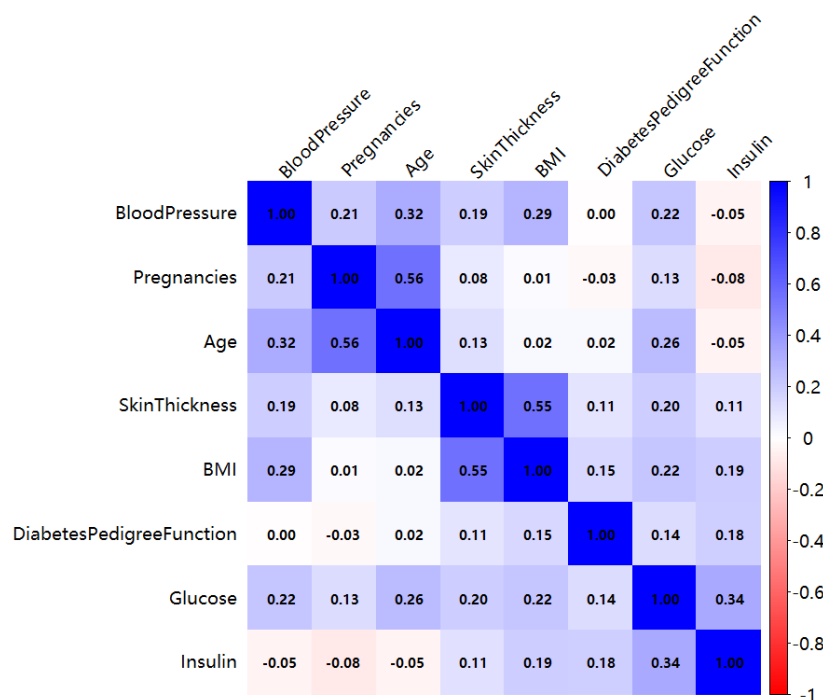


图 12: Logistic 线性回归解释变量相关性热力图

2.2 回归系数的估计

```

> # 准备数据
> if (is.numeric(diabetes_data$Outcome)) {
+   y <- as.numeric(diabetes_data$Outcome)
+ } else {
+   y <- as.numeric(as.character(diabetes_data$Outcome))
+ }
>
> X <- model.matrix(~ ., data = diabetes_data[, explanatory_vars])
> n <- nrow(X)
> p <- ncol(X)
>
> cat(sprintf("\n数据维度: n=%d, p=%d\n", n, p))

数据维度: n = 724, p = 9
> cat("解释变量:", paste(explanatory_vars, collapse = ", "), "\n")
解释变量: Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI,
          DiabetesPedigreeFunction, Age
>
> # Logistic线性回归的辅助函数
> sigmoid <- function(z) {
+   1 / (1 + exp(-z))
+ }
>
> log_likelihood <- function(beta, X, y) {
+   eta <- X %*% beta
+   p <- sigmoid(eta)
+   sum(y * log(p + 1e-10) + (1 - y) * log(1 - p + 1e-10))
+ }
>
> gradient <- function(beta, X, y) {
+   eta <- X %*% beta
+   p <- sigmoid(eta)
+   t(X) %*% (y - p)
+ }
>
> hessian <- function(beta, X, y) {
+   eta <- X %*% beta
+   p <- sigmoid(eta)
+   W <- diag(as.vector(p * (1 - p)))
+   -t(X) %*% W %*% X
+ }
>
> cat("\n2.1 Newton-Raphson算法迭代求解: \n")

2.1 Newton-Raphson算法迭代求解:
> cat("Newton-Raphson算法使用Hessian矩阵进行迭代更新\n")
Newton-Raphson算法使用Hessian矩阵进行迭代更新
>
> beta_nr <- rep(0, p)
> max_iter <- 100

```

```

> tolerance <- 1e-8
> nr_history <- data.frame(Iteration = integer(),
+                           LogLikelihood = numeric(),
+                           MaxGradient = numeric(),
+                           stringsAsFactors = FALSE)
>
> cat("\n迭代过程: \n")

  迭代过程:
> for (iter in 1:max_iter) {
+   grad <- gradient(beta_nr, X, y)
+   hess <- hessian(beta_nr, X, y)
+
+   # 检查Hessian矩阵是否可逆
+   if (any(is.na(hess)) || any(is.infinite(hess))) {
+     cat(sprintf("迭代%d: Hessian矩阵异常, 停止迭代\n", iter))
+     break
+   }
+
+   # 尝试求解
+   tryCatch({
+     delta <- solve(-hess, grad)
+     beta_nr <- beta_nr + delta
+
+     ll <- log_likelihood(beta_nr, X, y)
+     max_grad <- max(abs(grad))
+
+     nr_history <- rbind(nr_history, data.frame(
+       Iteration = iter,
+       LogLikelihood = ll,
+       MaxGradient = max_grad
+     ))
+
+     if (iter <= 5 || iter %% 10 == 0) {
+       cat(sprintf("迭代%d: 对数似然=%%.4f, 最大梯度=%%.6f\n",
+         iter, ll, max_grad))
+     }
+
+     if (max(abs(delta)) < tolerance) {
+       cat(sprintf("迭代%d: 收敛! 最大参数变化=%%.8f\n", iter, max(abs(delta))))
+       break
+     }
+   }, error = function(e) {
+     cat(sprintf("迭代%d: 求解失败-%s\n", iter, e$message))
+     break
+   })
+ }

  迭代 1: 对数似然 = -347.5320, 最大梯度 = 8611.500000
  迭代 2: 对数似然 = -336.0127, 最大梯度 = 1470.192141
  迭代 3: 对数似然 = -335.5030, 最大梯度 = 239.521958
  迭代 4: 对数似然 = -335.5014, 最大梯度 = 12.215703
  迭代 5: 对数似然 = -335.5014, 最大梯度 = 0.039913

```

```

    迭代 6: 收敛! 最大参数变化 = 0.00000000
>
> if (iter == max_iter) {
+   cat("警告: 达到最大迭代次数, 可能未完全收敛\n")
+ }
>
> cat("\nNewton-Raphson算法最终估计结果: \n")

    Newton-Raphson算法最终估计结果:
> names(beta_nr) <- colnames(X)
> print(round(beta_nr, 6))

              [,1]
(Intercept) -9.076802
Pregnancies  0.115861
Glucose       0.036889
BloodPressure -0.010538
SkinThickness 0.004851
Insulin      -0.001122
BMI           0.091086
DiabetesPedigreeFunction 1.006664
Age           0.015713
attr(,"names")
[1] "(Intercept)"      "Pregnancies"      "Glucose"
[4] "BloodPressure"    "SkinThickness"    "Insulin"
[7] "BMI"              "DiabetesPedigreeFunction" "Age"
>
> # 使用Newton-Raphson的结果作为最终估计
> beta_final <- beta_nr
>
> cat("\n2.2 回归方程: \n")

2.2 回归方程:
> cat("使用Newton-Raphson算法, Logistic线性回归方程为: \n")
    使用Newton-Raphson算法, Logistic线性回归方程为:
> cat("\nlogit(P(Y=1|X)) = ")

    logit(P(Y=1|X)) = > eq_parts <- c()
> for (i in seq_along(beta_final)) {
+   var_name <- names(beta_final)[i]
+   coef_val <- beta_final[i]
+   if (i == 1) {
+     eq_parts <- c(eq_parts, sprintf("%.4f", coef_val))
+   } else {
+     if (coef_val >= 0) {
+       eq_parts <- c(eq_parts, sprintf("+%.4f%s", coef_val, var_name))
+     } else {
+       eq_parts <- c(eq_parts, sprintf("-%.4f%s", abs(coef_val), var_name))
+     }
+   }
+ }
> cat(paste(eq_parts, collapse = "\n"))
-9.0768 + 0.1159*Pregnancies + 0.0369*Glucose - 0.0105*BloodPressure + 0.0049*SkinThickness -

```

```
0.0011*Insulin + 0.0911*BMI + 1.0067*DiabetesPedigreeFunction + 0.0157*Age> cat("\n\n其中\n\n
logit(p)=log(p/(1-p))\n")
```

其中 $\text{logit}(p) = \log(p/(1-p))$

```
>
> cat("\n2.3\n使用R内置glm函数验证:\n")
```

2.3 使用R内置glm函数验证:

```
> glm_model <- glm(Outcome ~ ., data = diabetes_data, family = binomial(link = "logit"))
> beta_glm <- coef(glm_model)
>
> beta_validation <- data.frame(
+   Variable = names(beta_final),
+   Fisher_Scoring = round(beta_final, 6),
+   GLM_Result = round(beta_glm, 6),
+   Difference = round(abs(beta_final - beta_glm), 6)
+ )
> print(beta_validation)
```

	Variable	Fisher_Scoring	GLM_Result	Difference
(Intercept)	(Intercept)	-9.076802	-9.076802	0
Pregnancies	Pregnancies	0.115861	0.115861	0
Glucose	Glucose	0.036889	0.036889	0
BloodPressure	BloodPressure	-0.010538	-0.010538	0
SkinThickness	SkinThickness	0.004851	0.004851	0
Insulin	Insulin	-0.001122	-0.001122	0
BMI	BMI	0.091086	0.091086	0
DiabetesPedigreeFunction	DiabetesPedigreeFunction	1.006664	1.006664	0
Age	Age	0.015713	0.015713	0

```
>
> max_diff_glm <- max(beta_validation$Difference)
> cat(sprintf("\n\n与glm函数结果的最大差异:%.8f\n", max_diff_glm))
```

与glm函数结果的最大差异: 0.00000000

```
> if (max_diff_glm < 1e-4) {
+   cat("估计结果与R内置函数高度一致, 验证通过\n")
+ } else {
+   cat("估计结果与R内置函数存在差异, 需要检查\n")
+ }
```

估计结果与R内置函数高度一致, 验证通过

2.3 回归系数的显著性检验

2.3.1 回归系数整体显著性的广义似然比检验

3.1 回归关系整体的显著性检验 (广义似然比检验):

```
>
> # 零模型 (只有截距项)
> null_model <- glm(Outcome ~ 1, data = diabetes_data, family = binomial)
> null_deviance <- null_model$deviance
```

```

> null_df <- null_model$df.residual
>
> # 完整模型
> full_deviance <- glm_model$deviance
> full_df <- glm_model$df.residual
>
> # 广义似然比统计量
> lr_statistic <- null_deviance - full_deviance
> lr_df <- null_df - full_df
> lr_p_value <- 1 - pchisq(lr_statistic, lr_df)
>
> cat(sprintf("零模型偏差: %.4f (自由度: %d)\n", null_deviance, null_df))
零模型偏差: 931.9373 (自由度: 723)
> cat(sprintf("完整模型偏差: %.4f (自由度: %d)\n", full_deviance, full_df))
完整模型偏差: 671.0027 (自由度: 715)
> cat(sprintf("似然比统计量: %.4f (自由度: %d)\n", lr_statistic, lr_df))
似然比统计量: 260.9346 (自由度: 8)
> cat(sprintf("p值: %.6f\n", lr_p_value))
p值: 0.000000
>
> if (lr_p_value < 0.001) {
+   cat("结论: p < 0.001, 回归关系高度显著\n")
+ } else if (lr_p_value < 0.01) {
+   cat("结论: p < 0.01, 回归关系非常显著\n")
+ } else if (lr_p_value < 0.05) {
+   cat("结论: p < 0.05, 回归关系显著\n")
+ } else {
+   cat("结论: p >= 0.05, 回归关系不显著\n")
+ }
结论: p < 0.001, 回归关系高度显著

```

2.3.2 单个回归系数显著性的 Wald 检验

```

> cat("\n3.2 单个回归系数的显著性检验 (Wald 检验) : \n")

3.2 单个回归系数的显著性检验 (Wald 检验) :
>
> coef_summary <- summary(glm_model)$coefficients
> wald_results <- data.frame(
+   Variable = rownames(coef_summary),
+   Coefficient = coef_summary[, 1],
+   StdError = coef_summary[, 2],
+   Z_Statistic = coef_summary[, 3],
+   P_Value = coef_summary[, 4],
+   Significant = ifelse(coef_summary[, 4] < 0.05, "是", "否")
+ )
>
> cat("\nWald 检验结果: \n")

Wald 检验结果:
> wald_results_display <- wald_results

```

```

> wald_results_display[, c("Coefficient", "StdError", "Z_Statistic", "P_Value")] <-
+   round(wald_results[, c("Coefficient", "StdError", "Z_Statistic", "P_Value")], 6)
> print(wald_results_display)

```

	Variable	Coefficient	StdError	Z_Statistic
(Intercept)	(Intercept)	-9.076802	0.829695	-10.939920
Pregnancies	Pregnancies	0.115861	0.033533	3.455151
Glucose	Glucose	0.036889	0.003880	9.506365
BloodPressure	BloodPressure	-0.010538	0.008718	-1.208855
SkinThickness	SkinThickness	0.004851	0.013405	0.361849
Insulin	Insulin	-0.001122	0.000849	-1.322489
BMI	BMI	0.091086	0.018477	4.929762
DiabetesPedigreeFunction	DiabetesPedigreeFunction	1.006664	0.310005	3.247255
Age	Age	0.015713	0.009869	1.592197

```


```

	P_Value	Significant
(Intercept)	0.000000	是
Pregnancies	0.000550	是
Glucose	0.000000	是
BloodPressure	0.226718	否
SkinThickness	0.717465	否
Insulin	0.186005	否
BMI	0.000001	是
DiabetesPedigreeFunction	0.001165	是
Age	0.111340	否

```

>
> cat("\n显著性总结: \n")

显著性总结:
> sig_vars <- wald_results$Variable[wald_results$P_Value < 0.05]
> non_sig_vars <- wald_results$Variable[wald_results$P_Value >= 0.05]
>
> cat(sprintf("显著变量(p<0.05): %d个\n", length(sig_vars)))
显著变量 (p < 0.05): 5 个
> if (length(sig_vars) > 0) {
+   for (var in sig_vars) {
+     p_val <- wald_results$P_Value[wald_results$Variable == var]
+     cat(sprintf("- %s (p = %.6f)\n", var, p_val))
+   }
+ }
- (Intercept) (p = 0.000000)
- Pregnancies (p = 0.000550)
- Glucose (p = 0.000000)
- BMI (p = 0.000001)
- DiabetesPedigreeFunction (p = 0.001165)
>
> cat(sprintf("不显著变量(p>=0.05): %d个\n", length(non_sig_vars)))
不显著变量 (p >= 0.05): 4 个
> if (length(non_sig_vars) > 0) {
+   for (var in non_sig_vars) {
+     p_val <- wald_results$P_Value[wald_results$Variable == var]
+     cat(sprintf("- %s (p = %.6f)\n", var, p_val))
+   }
+ }

```

```

- BloodPressure (p = 0.226718)
- SkinThickness (p = 0.717465)
- Insulin (p = 0.186005)
- Age (p = 0.111340)
>
> # 保存不显著变量列表供第五部分使用
> non_significant_vars <- non_sig_vars[non_sig_vars != "(Intercept)"]

```

2.3.3 回归系数的 95% 置信区间

```

> cat("\n3.3 回归系数的95%置信区间: \n")

3.3 回归系数的95%置信区间:
> coef_ci <- confint(glm_model, level = 0.95)
Waiting for profiling to be done...
> ci_results <- data.frame(
+   Variable = rownames(coef_ci),
+   Lower_CI = round(coef_ci[, 1], 6),
+   Upper_CI = round(coef_ci[, 2], 6)
+ )
> print(ci_results)

```

	Variable	Lower_CI	Upper_CI
(Intercept)	(Intercept)	-10.758599	-7.501564
Pregnancies	Pregnancies	0.050716	0.182380
Glucose	Glucose	0.029496	0.044731
BloodPressure	BloodPressure	-0.027707	0.006536
SkinThickness	SkinThickness	-0.021101	0.031399
Insulin	Insulin	-0.002793	0.000547
BMI	BMI	0.055436	0.128004
DiabetesPedigreeFunction	DiabetesPedigreeFunction	0.406020	1.621972
Age	Age	-0.003684	0.035104

```

>
> # 检查置信区间是否包含0
> ci_contains_zero <- (ci_results$Lower_CI <= 0) & (ci_results$Upper_CI >= 0)
> ci_results$Contains_Zero <- ifelse(ci_contains_zero, "是", "否")
> cat("\n置信区间包含0的变量 (不显著): \n")

置信区间包含0的变量 (不显著):
> zero_ci_vars <- ci_results$Variable[ci_contains_zero]
> if (length(zero_ci_vars) > 0) {
+   for (var in zero_ci_vars) {
+     cat(sprintf("- %s: [%%.4f, %%.4f]\n",
+               var, ci_results$Lower_CI[ci_results$Variable == var],
+               ci_results$Upper_CI[ci_results$Variable == var]))
+   }
+ } else {
+   cat("无\n")
+ }

- BloodPressure: [-0.0277, 0.0065]
- SkinThickness: [-0.0211, 0.0314]
- Insulin: [-0.0028, 0.0005]

```

```
- Age: [-0.0037, 0.0351]
```

2.4 预测结果

2.4.1 预测值生成

```
> cat("\n4.1 生成预测值: \n")

4.1 生成预测值:
>
> # 预测概率
> predicted_prob <- predict(glm_model, type = "response")
> cat(sprintf("预测概率范围: \n", min(predicted_prob), max(predicted_prob)))
预测概率范围: [0.0111, 0.9949]
> cat(sprintf("预测概率均值: \n", mean(predicted_prob)))
预测概率均值: 0.3439
>
> # 预测类别 (使用0.5作为阈值)
> predicted_class <- ifelse(predicted_prob > 0.5, 1, 0)
> cat(sprintf("预测类别分布: \n", sum(predicted_class == 0), sum(predicted_class == 1)))
预测类别分布: 0类 = 531, 1类 = 193
>
> # 实际类别
> actual_class <- y
> cat("\n4.4 预测值列表: \n")

4.2 预测值列表:
>
> # 创建预测结果数据框
> prediction_results <- data.frame(
+   观测编号 = 1:n,
+   实际类别 = actual_class,
+   预测概率 = round(predicted_prob, 4),
+   预测类别 = predicted_class,
+   预测正确 = ifelse(actual_class == predicted_class, "是", "否")
+ )
>
> cat("\n前20个观测的预测结果: \n")

前20个观测的预测结果:
> print(head(prediction_results, 20))
观测编号 实际类别 预测概率 预测类别 预测正确
1         1         1  0.7244         1         是
2         2         0  0.0424         0         是
3         3         1  0.7973         1         是
4         4         0  0.0352         0         是
5         5         1  0.9076         1         是
6         6         0  0.1357         0         是
7         7         1  0.0552         0         否
```

```

9          8          1  0.7433          1      是
11         9          0  0.2150          0      是
12        10          1  0.8982          1      是
13        11          0  0.7896          1      否
14        12          1  0.6480          1      是
15        13          1  0.6237          1      是
17        14          1  0.3951          0      否
18        15          1  0.1798          0      否
19        16          0  0.3231          0      是
20        17          1  0.2259          0      否
21        18          0  0.4107          0      是
22        19          0  0.3013          0      是
23        20          1  0.9469          1      是
>
> cat("\n后20个观测的预测结果：\n")

    后20个观测的预测结果：
> print(tail(prediction_results, 20))
    观测编号 实际类别 预测概率 预测类别 预测正确
749        705          1  0.8328          1      是
750        706          1  0.5648          1      是
751        707          1  0.5461          1      是
752        708          0  0.2772          0      是
753        709          0  0.0914          0      是
754        710          1  0.7088          1      是
755        711          1  0.7248          1      是
756        712          1  0.4673          0      否
757        713          0  0.4899          0      是
758        714          1  0.3157          0      否
759        715          0  0.1561          0      是
760        716          1  0.9129          1      是
761        717          0  0.0833          0      是
762        718          1  0.9368          1      是
763        719          0  0.0723          0      是
764        720          0  0.3092          0      是
765        721          0  0.3030          0      是
766        722          0  0.1542          0      是
767        723          1  0.2744          0      否
768        724          0  0.0648          0      是
>
> # 统计预测正确率
> correct_predictions <- sum(prediction_results$预测正确 == "是")
> cat(sprintf("\n预测正确数量：%d/%d(%.2f%%)\n",
+             correct_predictions, n, 100 * correct_predictions / n))

    预测正确数量：564 / 724 (77.90%)
>
> # 按预测概率分组统计
> cat("\n4.3预测概率分布分析：\n")

4.3 预测概率分布分析：
> prob_ranges <- cut(predicted_prob, breaks = seq(0, 1, by = 0.1), include.lowest = TRUE)

```

```

> prob_distribution <- table(prob_ranges)
> cat("预测概率分布: \n")
  预测概率分布:
> print(prob_distribution)
prob_ranges
 [0,0.1] (0.1,0.2] (0.2,0.3] (0.3,0.4] (0.4,0.5] (0.5,0.6] (0.6,0.7] (0.7,0.8] (0.8,0.9]
      165       132        99        80        55        31        41        54        41
(0.9,1]
      26
>
> # 高置信度预测统计
> high_conf_predictions <- sum(predicted_prob > 0.8 | predicted_prob < 0.2)
> cat(sprintf("\n高置信度预测 (概率>0.8或<0.2): %d (%.2f%%)\n",
+           high_conf_predictions, 100 * high_conf_predictions / n))

  高置信度预测 (概率>0.8或<0.2): 364 (50.28%)
>
> # 保存完整预测结果 (可选, 如果数据量不大)
> cat("\n完整预测结果已保存在prediction_results数据框中\n")

  完整预测结果已保存在prediction_results数据框中
> cat(sprintf("总观测数: %d\n", nrow(prediction_results)))
  总观测数: 724

```

2.4.2 预测性能评估

```

> cat("\n4.4 预测性能评估指标: \n")
4.4 预测性能评估指标:
>
> # 混淆矩阵
> conf_matrix <- table(Actual = actual_class, Predicted = predicted_class)
> cat("\n混淆矩阵: \n")

  混淆矩阵:
> print(conf_matrix)
      Predicted
Actual    0    1
      0  423  52
      1  108 141
>
> # 计算各种性能指标
> TP <- conf_matrix[2, 2] # True Positive
> TN <- conf_matrix[1, 1] # True Negative
> FP <- conf_matrix[1, 2] # False Positive
> FN <- conf_matrix[2, 1] # False Negative
>
> accuracy <- (TP + TN) / (TP + TN + FP + FN)
> precision <- TP / (TP + FP)
> recall <- TP / (TP + FN)
> specificity <- TN / (TN + FP)
> f1_score <- 2 * (precision * recall) / (precision + recall)

```

```

>
> cat("\n分类性能指标: \n")

分类性能指标:
> cat(sprintf("准确率(Accuracy): %.4f\n", accuracy))
准确率 (Accuracy): 0.7790
> cat(sprintf("精确率(Precision): %.4f\n", precision))
精确率 (Precision): 0.7306
> cat(sprintf("召回率(Recall/Sensitivity): %.4f\n", recall))
召回率 (Recall/Sensitivity): 0.5663
> cat(sprintf("特异性(Specificity): %.4f\n", specificity))
特异性 (Specificity): 0.8905
> cat(sprintf("F1分数(F1-Score): %.4f\n", f1_score))
F1分数 (F1-Score): 0.6380
>
> # AUC 计算
> if (!require(ROCR, quietly = TRUE)) {
+   install.packages("ROCR")
+   library(ROCR)
+ }
>
> pred_obj <- prediction(predicted_prob, actual_class)
> auc_value <- performance(pred_obj, "auc")@y.values[[1]]
> cat(sprintf("AUC值(Area Under Curve): %.4f\n", auc_value))
AUC值 (Area Under Curve): 0.8439
>
> # ROC 曲线
> roc_curve <- performance(pred_obj, "tpr", "fpr")
>
> cat("\n4.5 绘制 ROC 曲线: \n")

4.5 绘制 ROC 曲线:
> par(family = "微软雅黑")
> plot(roc_curve,
+      main = "ROC 曲线",
+      xlab = "假阳性率(1-Specificity)",
+      ylab = "真阳性率(Sensitivity)",
+      col = "blue",
+      lwd = 2)
> abline(a = 0, b = 1, col = "red", lty = 2, lwd = 1.5)
> text(0.6, 0.2, sprintf("AUC= %.4f", auc_value), cex = 1.2, col = "darkblue")
>
> # 性能指标总结表
> performance_summary <- data.frame(
+   指标 = c("准确率", "精确率", "召回率", "特异性", "F1分数", "AUC"),
+   数值 = round(c(accuracy, precision, recall, specificity, f1_score, auc_value), 4)
+ )
>
> cat("\n预测性能指标总结: \n")

预测性能指标总结:
> print(performance_summary)

```

指标	数值
1 准确率	0.7790
2 精确率	0.7306
3 召回率	0.5663
4 特异性	0.8905
5 F1分数	0.6380
6 AUC	0.8439

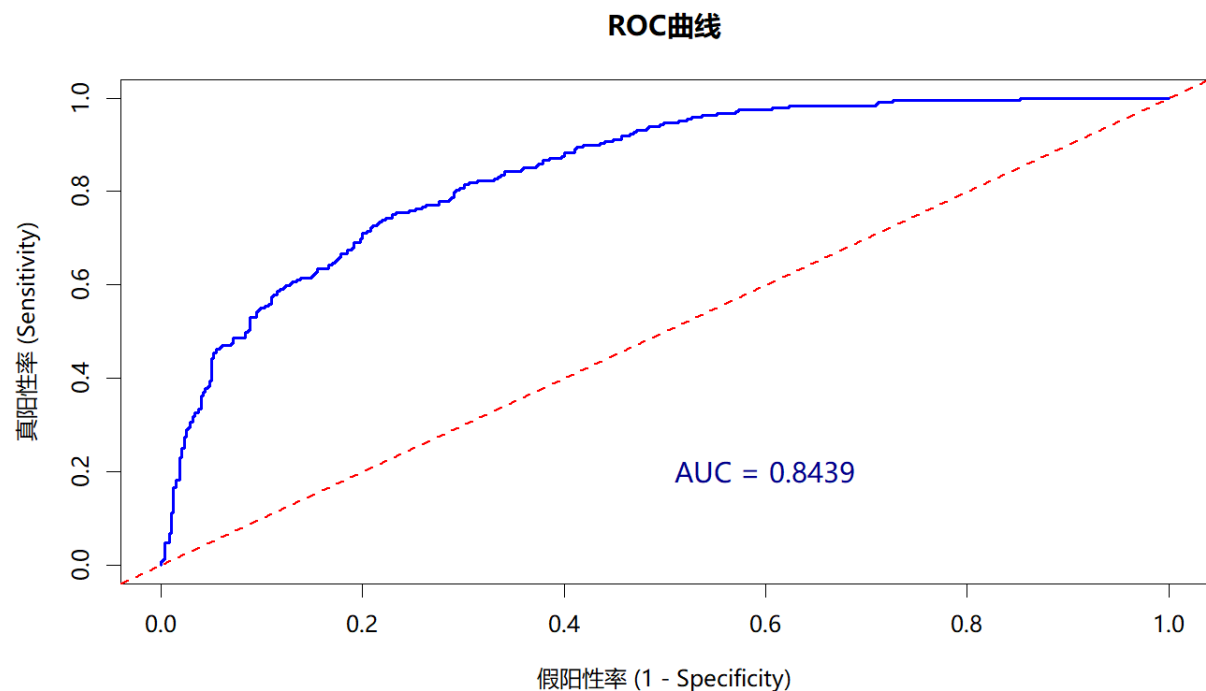


图 13: 模型的 ROC 曲线

2.5 模型改进

```

> if (length(non_significant_vars) > 0) {
+   cat(sprintf("\n5.1 识别不显著的回归系数分量: \n"))
+   cat(sprintf("发现 %d 个不显著变量 (p >= 0.05): \n", length(non_significant_vars)))
+   for (var in non_significant_vars) {
+     cat(sprintf("- %s \n", var))
+   }
+
+   cat("\n5.2 构建改进模型 (去除不显著变量): \n")
+
+   # 构建新的公式 (去除不显著变量, 但保留截距项)
+   significant_vars <- setdiff(explanatory_vars, non_significant_vars)
+
+   if (length(significant_vars) > 0) {
+     improved_formula <- as.formula(paste("Outcome ~", paste(significant_vars, collapse = "+")))
+   }
+ }

```

```

+   cat(sprintf("改进模型公式: %s\n", format(improved_formula)))
+
+   cat("\n5.3使用改进模型重新估计参数: \n")
+
+   # 使用Fisher Scoring重新估计
+   X_improved <- model.matrix(improved_formula, data = diabetes_data)
+   p_improved <- ncol(X_improved)
+
+   cat("\n使用Fisher Scoring算法重新估计: \n")
+   beta_improved <- rep(0, p_improved)
+
+   for (iter in 1:max_iter) {
+     grad <- gradient(beta_improved, X_improved, y)
+     fisher_info <- fisher_information(beta_improved, X_improved, y)
+
+     tryCatch({
+       delta <- solve(fisher_info, grad)
+       beta_improved <- beta_improved + delta
+
+       if (iter <= 3 || iter %% 10 == 0) {
+         ll <- log_likelihood(beta_improved, X_improved, y)
+         cat(sprintf("迭代 %d: 对数似然 = %.4f\n", iter, ll))
+       }
+
+       if (max(abs(delta)) < tolerance) {
+         cat(sprintf("迭代 %d: 收敛! \n", iter))
+         break
+       }
+     }, error = function(e) {
+       cat(sprintf("迭代失败: %s\n", e$message))
+       break
+     })
+   }
+
+   names(beta_improved) <- colnames(X_improved)
+   cat("\n改进模型参数估计结果: \n")
+   print(round(beta_improved, 6))
+
+   # 使用glm函数拟合改进模型
+   improved_glm <- glm(improved_formula, data = diabetes_data, family = binomial(link = "logit")
+ ))
+
+   cat("\n5.4改进模型的显著性检验: \n")
+
+   # 整体显著性检验
+   improved_null <- glm(Outcome ~ 1, data = diabetes_data, family = binomial)
+   improved_lr_stat <- improved_null$deviance - improved_glm$deviance
+   improved_lr_df <- improved_null$df.residual - improved_glm$df.residual
+   improved_lr_p <- 1 - pchisq(improved_lr_stat, improved_lr_df)
+
+   cat(sprintf("整体显著性检验: 似然比统计量 = %.4f, p值 = %.6f\n",
+     improved_lr_stat, improved_lr_p))

```

```

+
+ # 单个系数显著性检验
+ improved_coef_summary <- summary(improved_glm)$coefficients
+ improved_wald <- data.frame(
+   Variable = rownames(improved_coef_summary),
+   Coefficient = round(improved_coef_summary[, 1], 6),
+   StdError = round(improved_coef_summary[, 2], 6),
+   Z_Statistic = round(improved_coef_summary[, 3], 6),
+   P_Value = round(improved_coef_summary[, 4], 6)
+ )
+
+ cat("\n改进模型Wald检验结果: \n")
+ print(improved_wald)
+
+ cat("\n5.5模型比较: \n")
+
+ comparison_table <- data.frame(
+   Model = c("完整模型", "改进模型"),
+   Variables = c(length(explanatory_vars), length(significant_vars)),
+   AIC = round(c(AIC(glm_model), AIC(improved_glm)), 4),
+   BIC = round(c(BIC(glm_model), BIC(improved_glm)), 4),
+   Deviance = round(c(glm_model$deviance, improved_glm$deviance), 4),
+   Pseudo_R2 = round(c(1 - glm_model$deviance/glm_model$null.deviance,
+     1 - improved_glm$deviance/improved_glm$null.deviance), 4)
+ )
+
+ print(comparison_table)
+
+ cat("\n5.6改进模型的回归方程: \n")
+ cat("logit(P(Y=1|X))=")
+ improved_eq_parts <- c()
+ for (i in seq_along(beta_improved)) {
+   var_name <- names(beta_improved)[i]
+   coef_val <- beta_improved[i]
+   if (i == 1) {
+     improved_eq_parts <- c(improved_eq_parts, sprintf("%.4f", coef_val))
+   } else {
+     if (coef_val >= 0) {
+       improved_eq_parts <- c(improved_eq_parts, sprintf("+%.4f*%s", coef_val, var_name))
+     } else {
+       improved_eq_parts <- c(improved_eq_parts, sprintf("-%.4f*%s", abs(coef_val), var_name))
+     }
+   }
+ }
+ cat(paste(improved_eq_parts, collapse = "+"))
+ cat("\n")
+
+ cat("\n5.7改进效果总结: \n")
+ aic_improvement <- AIC(glm_model) - AIC(improved_glm)
+ bic_improvement <- BIC(glm_model) - BIC(improved_glm)

```

```

+   cat(sprintf("AIC改善: %.4f (降低表示改进)\n", aic_improvement))
+   cat(sprintf("BIC改善: %.4f (降低表示改进)\n", bic_improvement))
+   cat(sprintf("变量数量: 从 %d 减少到 %d\n",
+               length(explanatory_vars), length(significant_vars)))
+
+   if (aic_improvement > 0 && bic_improvement > 0) {
+     cat(" 改进模型在AIC和BIC指标上均优于完整模型\n")
+   } else if (aic_improvement > 0) {
+     cat(" 改进模型在AIC指标上优于完整模型\n")
+   } else {
+     cat(" 改进模型在某些指标上未优于完整模型，但模型更简洁\n")
+   }
+
+ } else {
+   cat("警告: 所有变量都不显著，无法构建改进模型\n")
+ }
+
+ } else {
+   cat("\n5.1 不显著变量检查: \n")
+   cat("所有回归系数分量均显著 (p < 0.05)，无需去除变量\n")
+   cat("当前模型即为最优模型\n")
+ }

```

5.1 识别不显著的回归系数分量：

发现 4 个不显著变量 ($p \geq 0.05$):

```

- BloodPressure
- SkinThickness
- Insulin
- Age

```

5.2 构建改进模型（去除不显著变量）：

改进模型公式: $\text{Outcome} \sim \text{Pregnancies} + \text{Glucose} + \text{BMI} + \text{DiabetesPedigreeFunction}$

5.3 使用改进模型重新估计参数：

使用Fisher Scoring算法重新估计：

```

迭代 1: 对数似然 = -349.3342
迭代 2: 对数似然 = -338.5391
迭代 3: 对数似然 = -338.1147
迭代 6: 收敛!

```

改进模型参数估计结果：

```

               [,1]
(Intercept)    -9.000047
Pregnancies      0.143159
Glucose          0.035845
BMI              0.084367
DiabetesPedigreeFunction 0.973367
attr(,"names")
[1] "(Intercept)"          "Pregnancies"          "Glucose"
[4] "BMI"                  "DiabetesPedigreeFunction"

```

5.4 改进模型的显著性检验：

整体显著性检验：似然比统计量 = 255.7100, p值 = 0.000000

改进模型Wald检验结果：

	Variable	Coefficient	StdError	Z_Statistic
(Intercept)	(Intercept)	-9.000047	0.717019	-12.552035
Pregnancies	Pregnancies	0.143159	0.028342	5.051087
Glucose	Glucose	0.035845	0.003512	10.205447
BMI	BMI	0.084367	0.014981	5.631654
DiabetesPedigreeFunction	DiabetesPedigreeFunction	0.973367	0.304351	3.198175
	P_Value			
(Intercept)		0.000000		
Pregnancies		0.000000		
Glucose		0.000000		
BMI		0.000000		
DiabetesPedigreeFunction		0.001383		

5.5 模型比较：

	Model	Variables	AIC	BIC	Deviance	Pseudo_R2
1	完整模型	8	689.0027	730.2658	671.0027	0.2800
2	改进模型	4	686.2274	709.1513	676.2274	0.2744

5.6 改进模型的回归方程：

$\text{logit}(P(Y=1|X)) = -9.0000 + 0.1432 * \text{Pregnancies} + 0.0358 * \text{Glucose} + 0.0844 * \text{BMI} + 0.9734 * \text{DiabetesPedigreeFunction}$

5.7 改进效果总结：

AIC改善：2.7754（降低表示改进）

BIC改善：21.1145（降低表示改进）

变量数量：从 8 减少到 4

改进模型在AIC和BIC指标上均优于完整模型

3 课后习题 2.2、2.3

见下页